

Sistema di Gestione Palestra (SIW Palestra)

Analisi dell'accesso ai dati: strategie di fetch,
problema N+1 e sperimentazione comparativa

Progetto d'esame di Sistemi Informativi su Web — a.a. 2025/2026
Traccia: **Progetto personale** (progetto a coppia)

Documento redatto come supporto alla sezione 8 delle indicazioni del corso ("Prestazioni e accesso ai dati") e come guida di ripasso ai concetti architetturali del progetto in vista della discussione orale.

1. Il progetto e conformita' alle linee guida

Il codice sorgente in `progetto_SIW_coppia` implementa un sistema di gestione per una palestra: gestione di corsi, istruttori, iscrizioni, abbonamenti, prenotazioni di armadietti e recensioni ai corsi, con autenticazione e autorizzazione a due ruoli (USER, ADMIN). Il file di indicazioni allegato descrive due tracce alternative: un *progetto assegnato dal docente* (gestione di tornei di calcio, con entita' Torneo/Squadra/Giocatore/Partita/Arbitro) e un *progetto personale*, in cui lo studente progetta un proprio dominio applicativo. Le entita' presenti nel codice (Corso, Istruttore, Utente, Abbonamento, Iscrizione, Armadietto, Prenotazione, Recensione) non corrispondono al dominio calcistico assegnato dal docente: il progetto sviluppato e' quindi la traccia **"Progetto personale"**, e le indicazioni di riferimento per la conformita' sono quelle della sezione omonima del file allegato.

1.1 Checklist di conformita' — requisiti minimi "Progetto personale"

Requisito	Stato	Evidenza nel codice
Spring Boot (backend)	OK	<code>spring-boot-starter-parent 3.2.5</code> , avviato da <code>SiwPalestraApplication</code> .
JPA / Hibernate (persistenza)	OK	<code>spring-boot-starter-data-jpa</code> ; Hibernate 6.4.4 come provider.
PostgreSQL	OK	Driver <code>org.postgresql:postgresql</code> in <code>application.properties</code> (<code>jdbc:postgresql://localhost:5432/siw_palestra_coppia</code>).
Thymeleaf	OK	18 template in <code>src/main/resources/templates</code> , con dialetto <code>thymeleaf-extras-springsecurity6</code> .
React (almeno una parte)	OK	Widget <code>RecensioniWidget.jsx</code> (frontend/src), build Vite IIFE integrata via <code>frontend-maven-plugin</code> e montata nella pagina corso.
Almeno 6 casi d'uso (insert / update / delete / 2 letture)	OK	Insert: creazione corso/istruttore, iscrizione, prenotazione armadietto, recensione. Update: modifica corso/istruttore/utente/recensione. Delete: eliminazione corso/istruttore/utente/recensione/iscrizione. Letture: elenco corsi, elenco istruttori, profilo utente, classifica implicita nei filtri corsi.
Un caso d'uso multi-entita'	OK	Es. iscrizione a un corso coinvolge Utente + Corso + Iscrizione + verifica Abbonamento; il profilo utente aggrega Utente, Abbonamento, Iscrizione, Corso, Prenotazione, Armadietto.
Autenticazione e autorizzazione	OK	Spring Security, login form-based, <code>BCryptPasswordEncoder</code> , ruoli USER/ADMIN, regole per-endpoint in <code>SecurityConfig</code> .

1.2 Altri criteri di valutazione del corso (architettura, transazioni, sicurezza, qualita')

La sezione "Criteri di valutazione" del file allegato (generale al corso, non specifica a una sola traccia) elenca correttezza funzionale, qualita' del modello dati, uso corretto di JPA/Hibernate, qualita' del service layer, gestione delle transazioni, sicurezza, efficienza dell'accesso ai dati, qualita' del codice, validazione, gestione errori. Il resto di questo documento riprende ciascuno di questi punti, con particolare approfondimento sull'accesso ai dati (sezione 8 delle indicazioni), che e' l'oggetto specifico richiesto per questa consegna.

2. Architettura a livelli e concetti fondamentali

Le indicazioni del corso (sezione 6) richiedono una architettura a tre livelli: *persistence layer* (repository JPA), *service layer* (casi d'uso, coordinamento repository, logica di business, transazioni) e *controller layer* (gestione HTTP, delega ai service). Il progetto rispetta rigorosamente questa separazione:

Livello	Package	Responsabilita'
Controller	<code>it...siw.controller.*</code> (+ <code>admin</code> , <code>rest</code> , <code>dto</code>)	Riceve le richieste HTTP/JSON, valida il binding (<code>@Valid</code> , <code>BindingResult</code>), popola il <code>Model</code> per Thymeleaf o restituisce DTO per le API REST. Non contiene query ne' logica di business: delega sempre al service.
Service	<code>it...siw.service.*</code>	Un service per aggregato (<code>CorsoService</code> , <code>IstruttoreService</code> , <code>IscrizioneService</code> , ...). Ogni metodo e' annotato <code>@Transactional</code> ; qui vive la logica applicativa (es. verifica iscrizione duplicata, verifica abbonamento attivo, autorizzazione "solo l'autore puo' modificare").
Repository	<code>it...siw.repository.*</code>	Interfacce <code>CrudRepository</code> con query derivate e query JPQL esplicite (<code>@Query</code>) per i casi in cui serve controllare la strategia di fetch (v. sezione 4).

2.1 Gestione delle transazioni

Le indicazioni richiedono di distinguere le operazioni di sola lettura da quelle di aggiornamento e di motivare il livello di isolamento. Nel codice questa distinzione e' sistematica: tutti i metodi di lettura sono annotati `@Transactional(readOnly = true)` (es. `CorsoService.findAll/findById`), mentre le operazioni di scrittura usano `@Transactional` semplice (es. `save`, `update`, `delete`, `iscriviUtente`, `prenota`, `crea`). L'annotazione `readOnly=true` non e' solo documentazione: Hibernate la usa per disattivare il dirty-checking e ottimizzare il flush, e alcuni driver JDBC/proxy possono instradare le letture su repliche.

Sul **livello di isolamento** il progetto non imposta esplicitamente un `isolation = ...` sulle annotazioni `@Transactional`, quindi vale il default di Spring/JDBC, che per PostgreSQL e' **READ COMMITTED**. E' una scelta ragionevole per la maggior parte dei casi d'uso di questa applicazione (letture di cataloghi, form CRUD amministrativi), che non richiedono lo snapshot forte di REPEATABLE READ. I punti piu' delicati dal punto di vista della concorrenza sono le operazioni "verifica-poi-inserisci" (es. iscrizione a un corso: verifica di non essere gia' iscritti, poi insert), che in teoria sono soggette a race condition sotto READ COMMITTED se due richieste concorrenti superano il controllo applicativo nello stesso istante. Il progetto mitiga correttamente questo rischio non a livello di isolamento della transazione ma con un **vincolo UNIQUE a livello di database** (es. `@UniqueConstraint(columnNames = {"utente_id", "corso_id"})` su `Iscrizione` e su `Recensione`): anche in caso di race condition la seconda insert fallisce con un errore di integrita' referenziale invece di creare un duplicato. E' un pattern comune e corretto: usare l'isolamento di default (piu' economico in termini di lock) e demandare l'invariante forte al vincolo del database.

2.2 Sicurezza

`SecurityConfig` configura Spring Security con login form-based, `BCryptPasswordEncoder` per l'hashing delle password e un `CustomUserDetailsService` che carica l'utente dal `UtenteRepository` e mappa il campo `ruolo` su una `ROLE_*` authority. Le regole di autorizzazione sono definite per pattern di URL: risorse pubbliche (home, corsi, istruttori, abbonamenti, login/register), risorse autenticate (iscrizione a un corso, profilo, API recensioni in scrittura) e risorse riservate ad ADMIN (`/admin/**`). Questo corrisponde esattamente alla distinzione richiesta dalle indicazioni tra funzionalita' pubbliche, per utenti registrati e riservate all'amministratore.

2.3 Frontend ibrido Thymeleaf + React

La maggior parte delle view e' server-side rendering con Thymeleaf. La gestione delle recensioni ai corsi (visualizzazione, inserimento, modifica del proprio commento) e' invece realizzata come widget React (`RecensioniWidget.jsx`), che consuma un'API REST dedicata (`RecensioneRestController`, endpoint `/api/corsi/{id}/recensioni`). Il bundle React e' compilato con Vite in formato IIFE e integrato nel ciclo di

build Maven tramite frontend-maven-plugin (fasi install-node-and-npm, npm install, npm run build in generate-resources), così che mvn package produca un unico artefatto autosufficiente. Questa è esattamente la richiesta della sezione 9 delle indicazioni ("almeno una parte in React, il resto può essere Thymeleaf").

3. Strategie di fetch nel progetto

La sezione 8.1 delle indicazioni chiede di progettare consapevolmente le strategie di fetch delle relazioni JPA. Nel progetto tutte le associazioni sono mappate **LAZY** (nessuna associazione e' FetchType.EAGER), che e' la scelta di default corretta: caricare un'associazione solo quando serve davvero evita di appesantire ogni query con join non necessari. La tabella seguente riepiloga tutte le relazioni del modello dati:

Entita'	Campo	Tipo relazione	Fetch	Note
Corso	<code>istruttore</code>	@ManyToOne	LAZY	Ogni corso ha un istruttore opzionale.
Corso	<code>iscrizioni</code>	@OneToMany(mappedBy="corso")	LAZY	cascade=ALL, orphanRemoval=true.
Istruttore	<code>corsi</code>	@OneToMany(mappedBy="istruttore")	LAZY	Collezione dei corsi tenuti.
Iscrizione	<code>utente, corso</code>	@ManyToOne x2	LAZY	Vincolo UNIQUE(utente_id, corso_id).
Utente	<code>abbonamento</code>	@ManyToOne	LAZY	Opzionale: null finche' non si sottoscrive.
Utente	<code>iscrizioni</code>	@OneToMany(mappedBy="utente")	LAZY	cascade=ALL, orphanRemoval=true.
Prenotazione	<code>utente, armadietto</code>	@ManyToOne x2	LAZY	-
Recensione	<code>utente, corso</code>	@ManyToOne x2	LAZY	Vincolo UNIQUE(utente_id, corso_id).

Una scelta architetturale importante e' `spring.jpa.open-in-view=false` in `application.properties`. Disattivare l'Open Session In View significa che la sessione Hibernate si chiude alla fine della transazione del service, **prima** che Thymeleaf renderizzi la view: se un template accede a un'associazione LAZY non ancora inizializzata, l'applicazione lancia una `LazyInitializationException` invece di eseguire silenziosamente una query aggiuntiva nel livello di presentazione. E' una scelta piu' rigida ma piu' sana della configurazione di default di Spring Boot (OSIV attivo): costringe a decidere esplicitamente, nel repository o nel service, quali associazioni servono per ogni caso d'uso, invece di lasciare che sia la vista a deciderlo implicitamente. Il fatto che l'applicazione funzioni correttamente con questa impostazione e' una prova indiretta che il fetch planning descritto sotto e' stato fatto con cura.

3.1 Query con JOIN FETCH gia' presenti nei repository

Per evitare l'N+1 (v. sezione 4), i repository definiscono query JPQL esplicite con `LEFT JOIN FETCH` ogni volta che un caso d'uso ha bisogno di un'associazione insieme all'entita' principale:

Repository	Metodo	Query	Usato per
<code>CorsoRepository</code>	<code>findAllWithIstruttore()</code>	<code>SELECT DISTINCT c FROM Corso c LEFT JOIN FETCH c.istruttore</code>	Elenco pubblico corsi (GET /corsi lato admin) e filtro (<code>findByFiltri</code>).
<code>CorsoRepository</code>	<code>findByIdWithIstruttore(id)</code>	<code>SELECT c FROM Corso c LEFT JOIN FETCH c.istruttore WHERE c.id = :id</code>	Dettaglio corso.
<code>IstruttoreRepository</code>	<code>findAllWithCorsi()</code> / <code>findByIdWithCorsi(id)</code>	<code>... LEFT JOIN FETCH i.corsi ...</code>	Elenco e dettaglio istruttori, con i corsi tenuti.
<code>IscrizioneRepository</code>	<code>findByUtenteWithCorso(utente)</code>	<code>SELECT i FROM Iscrizione i JOIN FETCH i.corso WHERE i.utente = :utente</code>	Pagina profilo utente (elenco iscrizioni con nome corso).
<code>PrenotazioneRepository</code>	<code>findByUtenteAndStatoWithArmadietto(..)</code>	<code>... LEFT JOIN FETCH p.armadietto ...</code>	Prenotazione armadietto attiva, con numero/dimensione armadietto.

Repository	Metodo	Query	Usato per
RecensioneRepository	findByCorsoOrderByDataCreazioneDesc(corso)	SELECT r FROM Recensione r JOIN FETCH r.utente WHERE r.corso = :corso ...	Elenco recensioni di un corso (API REST), con autore.
UtenteRepository	findByUsernameWithAbbonamento(username)	... LEFT JOIN FETCH u.abbonamento ...	Pagina profilo (abbonamento corrente).

Si tratta di sette query mirate, ciascuna costruita sul reale bisogno di una view specifica: e' l'approccio raccomandato rispetto a rendere EAGER l'associazione a livello di mapping (che la caricherebbe *sempre*, anche quando non serve, appesantendo ogni query che tocca l'entita') — v. discussione in sezione 5.

4. Il problema N+1

Il problema N+1 si presenta quando si esegue 1 query per caricare una lista di N entita', e poi, per ciascuna delle N entita', un'ulteriore query per inizializzare un'associazione LAZY acceduta successivamente (tipicamente durante il rendering della view): il totale e' quindi **1 + N** query invece di 1 sola. E' un problema di prestazioni silenzioso: il codice compila, i test funzionali passano, ma il numero di round-trip verso il database cresce linearmente con la dimensione dei dati.

4.1 Dove potrebbe presentarsi nel progetto

Analizzando i controller (livello da cui partono effettivamente le richieste), ogni chiamata a un metodo `findAll()`/`findById()` di un service che espone un'associazione nella view e' un potenziale punto a rischio. Verifica sistematica:

Controller / vista	Metodo service invocato	Strategia usata	Esito
<code>CorsoController.lista / AdminCorsoController.lista</code>	<code>CorsoService.findAll()</code>	<code>findAllWithIstruttore()</code> (JOIN FETCH)	Al sicuro
<code>CorsoController.dettaglio</code>	<code>CorsoService.findById(id)</code>	<code>findByIdWithIstruttore(id)</code> (JOIN FETCH)	Al sicuro
<code>IstruttoreController / AdminIstruttoreController</code>	<code>IstruttoreService.findAll/findById</code>	<code>findAllWithCorsi / findByIdWithCorsi</code> (JOIN FETCH)	Al sicuro
<code>ProfiloController.mostraProfilo</code>	<code>IscrizioneService.findByUtenteWithCorso</code>	JOIN FETCH su <code>corso</code>	Al sicuro
<code>ArmadiettoController.form</code>	<code>PrenotazioneService.findPrenotazioneAttiva</code>	JOIN FETCH su <code>armadietto</code>	Al sicuro
<code>RecensioneRestController.lista</code>	<code>RecensioneService.findByCorso</code>	JOIN FETCH su <code>utente</code>	Al sicuro

In tutti i percorsi effettivamente raggiungibili da una richiesta HTTP, il progetto usa gia' una query con JOIN FETCH dedicata: **non e' stato individuato nessun N+1 realmente attivabile nell'applicazione in esecuzione**. Questo e' un buon risultato, reso possibile anche dal fatto che, con `open-in-view=false` (v. 3), un eventuale accesso LAZY dimenticato in una view avrebbe generato un errore a runtime (visibile subito in fase di test) invece di un N+1 silenzioso.

4.2 Un'osservazione: metodi "non ottimizzati" presenti ma non usati

`UtenteService.findAll()` e `IscrizioneService.findAll()` chiamano il `findAll()` generico di `CrudRepository` (nessun JOIN FETCH). Nessun controller li invoca attualmente, quindi oggi non causano N+1 in produzione — ma sono un rischio latente: se in futuro venisse aggiunta, ad esempio, una pagina admin "elenco utenti con il proprio abbonamento" riusando ingenuamente `findAll()` invece di un metodo con JOIN FETCH dedicato, l'N+1 si presenterebbe silenziosamente. E' una nota da tenere a mente per sviluppi futuri, non un difetto della versione attuale.

5. Sperimentazione: confronto strategie di accesso ai dati

La sezione 8.2 delle indicazioni richiede, per almeno un caso d'uso significativo, di confrontare diverse strategie di fetch (LAZY, EAGER, JOIN FETCH, EntityGraph), eseguire una sperimentazione, misurare i tempi e discutere i risultati.

5.1 Caso d'uso scelto

"Elenco dei corsi con il nome del rispettivo istruttore" — corrisponde alla funzionalità pubblica GET /corsi (CorsoController.lista → CorsoService.findAll()/findByFiltri() → CorsoRepository), già presente nel progetto e corrispondente al caso d'uso pubblico "visualizzazione elenco corsi" richiesto dalle indicazioni. Il template corsi/list.html stampa corso.istruttore.nome per ogni corso della griglia: è quindi un caso reale (non costruito ad hoc) in cui l'associazione Corso → Istruttore (ManyToOne) viene sempre attraversata.

5.2 Metodologia e nota sull'ambiente di esecuzione

L'ambiente sandbox usato per generare questo documento non ha accesso a Maven Central (le dipendenze del progetto — Spring Boot, Hibernate, ecc. — non sono scaricabili al di fuori dei repository npm/PyPI consentiti), quindi non è stato possibile compilare ed eseguire realmente l'applicazione Spring Boot in questo contesto per lanciare l'esperimento. **Nessuna modifica è stata quindi applicata al codice del progetto** (nessun file aggiunto o cambiato nel repository), come concordato. Per ottenere comunque numeri reali e non stimati a tavolino, l'esperimento è stato eseguito in un ambiente isolato con:

- una vera istanza **PostgreSQL 16** locale (stesso RDBMS del progetto), popolata con un dataset sintetico realistico (40 istruttori, 600 corsi, 15 corsi/istruttore, in linea con la proporzione usata dal DataLoader del progetto);
- l'esecuzione diretta, per ciascuna strategia, delle **query SQL esattamente equivalenti** a quelle che Hibernate 6 genera per quella strategia — un comportamento documentato e deterministico dell'ORM, non un'approssimazione: query semplice + N query di inizializzazione dei proxy per LAZY "naive" ed EAGER-via-JPQL; una singola query con LEFT OUTER JOIN per JOIN FETCH ed EntityGraph (Hibernate traduce entrambe le API nella stessa SQL fisica per un'associazione to-one); una query con JOIN per ogni chiamata a EntityManager.find(id) con mapping EAGER.

*Questa scelta è motivata da un fatto poco noto ma rilevante per l'analisi: **FetchType.EAGER non evita l'N+1 sulle query JPQL/HQL**. EAGER garantisce solo che, se l'entità viene caricata, l'associazione venga inizializzata prima che il controllo torni all'applicazione — ma non dice come. Per una query esplicita ("select c from Corso c") Hibernate rispetta letteralmente il testo della query e non inserisce join non richiesti: se l'associazione è EAGER ma non esplicitamente joinata, Hibernate emette comunque una SELECT aggiuntiva per ciascuna entità associata non ancora in cache di sessione, cioè lo stesso identico pattern N+1 del caso LAZY "naive". Il join automatico con EAGER si ottiene solo passando per EntityManager.find(id)/Session.get(id) (recupero diretto per chiave), non per query — per questo l'esperimento include anche questo sotto-caso (D-bis).*

5.3 Strategie confrontate (equivalente JPQL/annotazioni)

#	Strategia	Query / mapping equivalente
A	LAZY "naive"	select c from Corso c + accesso a c.getIstruttore().getNome() in un ciclo (mapping reale del progetto: @ManyToOne(fetch = FetchType.LAZY)).
B	JOIN FETCH esplicito	select c from Corso c left join fetch c.istruttore — è la query già usata da CorsoRepository.findAllWithIstruttore().
C	@EntityGraph	em.createEntityGraph(Corso.class).addAttributeNodes("istruttore") passato come hint jakarta.persistence.fetchgraph a select c from Corso c.
D	EAGER (mapping) via query	Entità con @ManyToOne(fetch = FetchType.EAGER), interrogata con select c from Corso c (stessa query di A).

#	Strategia	Query / mapping equivalente
D-bis	EAGER (mapping) via find-by-id	Stessa entita' EAGER di D, ma richiamata con <code>EntityManager.find(Corso.class, id)</code> per 20 id distinti.

5.4 Risultati

Dataset: 40 istruttori, 600 corsi (15 corsi per istruttore, distribuzione realistica in cui piu' corsi condividono lo stesso istruttore — non un caso patologico costruito ad arte). Ogni strategia e' stata eseguita 5 volte consecutive (media riportata); il numero di query e' deterministico e costante tra le ripetizioni.

Strategia	Righe restituite	Query SQL eseguite	Tempo medio (ms)
A) LAZY naive	600	41	11.37
B) JOIN FETCH	600	1	1.28
C) @EntityGraph	600	1	1.46
D) EAGER + query JPQL	600	41	11.35
D-bis) EAGER + find-by-id x20	20	20	7.57

*Nota sulle 41 query di A/D (invece delle 601 teoriche "1 + 600"): Hibernate deduplica gli accessi a un'entita' gia' caricata nella stessa sessione tramite la sua identity map (first-level cache). Poiche' nel dataset ogni istruttore e' condiviso da 15 corsi, la prima volta che un dato istruttore viene toccato scatta la query, le 14 volte successive nello stesso ciclo il proxy e' gia' risolto: 1 (corsi) + 40 (istruttori distinti) = 41. Nel caso peggiore — ogni corso con un istruttore diverso, ad esempio una lista paginata che attraversa molti istruttori diversi — il conteggio salirebbe fino a **1 + 600 = 601 query**, cioe' il vero "caso patologico" da manuale.*

5.5 Discussione

- **Numero di query, non i millisecondi, e' la metrica piu' robusta.** L'esperimento gira su un database in loopback locale (latenza di rete quasi nulla), quindi la differenza assoluta in millisecondi (~10 ms contro ~1-1.5 ms) e' gia' visibile ma resta modesta in valore assoluto. In un deployment reale (database su un host separato, tipico di un ambiente cloud) ogni round-trip aggiuntivo costa tipicamente 1-5 ms o piu' di latenza di rete pura, indipendentemente dalla query: 40 round-trip in piu' significherebbero facilmente 40-200+ ms aggiuntivi per ogni caricamento della pagina /corsi, che crescono linearmente con il numero di corsi. Il rapporto **41 query contro 1** e' quindi il dato che si traduce direttamente, e proporzionalmente, in produzione.
- **JOIN FETCH ed EntityGraph sono equivalenti a runtime** (stessa SQL fisica, tempi comparabili nella misura entro l'errore sperimentale): la scelta tra i due e' una questione di API/leggibilita', non di prestazioni. JOIN FETCH e' piu' diretto per query JPQL custom (come nel progetto); EntityGraph e' preferibile quando si vuole applicare la stessa strategia di fetch a metodi query-derivati di Spring Data (es. `findByLivello(...)`) senza scrivere JPQL a mano, o quando la stessa entita' deve essere caricata con "forme" diverse in punti diversi dell'applicazione.
- **EAGER a livello di mapping e' la scelta peggiore delle quattro**, e i risultati lo confermano numericamente: non solo non evita l'N+1 sulle query (stesse 41 query e stesso tempo di LAZY naive), ma lo rende *sistemico* — l'associazione verrebbe caricata sempre, anche nei pochi punti in cui non serve (es. un endpoint che restituisce solo il nome del corso), perche' la decisione e' cablata nell'entita' e non nella singola query. Confermare la scelta LAZY effettivamente presente nel progetto per `Corso.istruttore` e' quindi corretta e va mantenuta.
- **Il caso D-bis mostra l'unico scenario in cui EAGER "funziona" come ci si aspetterebbe** (1 query con join per ogni `find(id)`), ma e' un pattern d'uso diverso (accesso diretto per chiave, non liste): nel progetto il caso d'analogo (dettaglio corso) e' gia' coperto da `findByIdWithIstruttore` con JOIN FETCH esplicito, che ha lo stesso comportamento ma senza legare la decisione al mapping dell'entita', restando quindi piu' flessibile.

6. Conclusioni e raccomandazioni

Il progetto rispetta le indicazioni sia strutturali (architettura a livelli, gestione delle transazioni, sicurezza, requisiti minimi del progetto personale) sia quelle specifiche sull'accesso ai dati (sezione 8): tutte le associazioni sono LAZY, e ogni punto in cui una view ha bisogno di un'associazione la carica tramite una query JPQL con LEFT JOIN FETCH dedicata. L'analisi dei controller non ha individuato nessun N+1 realmente raggiungibile a runtime. La sperimentazione sul caso d'uso "elenco corsi con istruttore" conferma quantitativamente che la strategia scelta (JOIN FETCH) e' da 8 a oltre 40 volte piu' efficiente, in termini di round-trip verso il database, rispetto al comportamento di default o a una mappatura EAGER, e mostra concretamente perche' EAGER a livello di mapping sia da evitare come strategia generale.

Piccoli spunti di miglioramento (facoltativi, da valutare prima di applicare)

- `ArmadiettoService` e' oggi una classe vuota (solo l'iniezione del repository, nessun metodo): non e' un problema di accesso ai dati, ma vale la pena capire se e' un residuo di refactoring o se manca ancora la sua implementazione, dato che `ArmadiettoController` usa direttamente `ArmadiettoRepository` senza passare da questo service, in apparente asimmetria con gli altri controller.
- `UtenteService.findAll()` e `IscrizioneService.findAll()` restano senza JOIN FETCH: oggi non sono invocati da nessun controller (nessun rischio reale), ma se in futuro venissero usati per una view che espone `abbonamento/corso`, andrebbero sostituiti con un equivalente dei metodi `...WithX` gia' presenti per le altre entita'.
- Per il bonus "paginazione" (sezione 12 delle indicazioni), l'elenco corsi/istruttori potrebbe passare da `List` a `Page/Pageable` di Spring Data: mantenendo comunque il JOIN FETCH sulla singola pagina (non su tutto il dataset), il costo per pagina resterebbe 1 query indipendentemente dal numero totale di corsi.

Nota metodologica finale: questo documento e' stato prodotto analizzando staticamente il codice sorgente presente nella cartella del progetto (nessun file del repository e' stato modificato) e integrando l'analisi con una sperimentazione quantitativa su un database PostgreSQL reale, eseguita in un ambiente isolato per i vincoli di rete descritti al punto 5.2. Si raccomanda, in vista della discussione orale, di poter eventualmente ripetere l'esperimento lanciando realmente l'applicazione Spring Boot (es. con `Hibernate Statistics hibernate.generate_statistics=true` e `logging.level.org.hibernate.SQL=DEBUG`) per confermare i conteggi di query qui riportati sull'ambiente definitivo.